

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Теоретические основы баз данных
Отчет о выполнении курсовой работы

**Реализация базы данных для предметной
области «Геомониторинг автопарка
каршеринга»**

Студент,
группы 5130201/40003

Адиатуллин Т. Р. _____

Руководитель,
доцент к.т.н., доцент

Попов С. Г. _____

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Реферат

Содержание

Введение	4
1 Постановка задачи	5
2 Аналитика	6
2.1 Описание предметной области	6
2.2 Цели функционирования базы данных	9
2.3 Сущности предметной области и их атрибуты	9
2.4 Перечень ролей	12
2.5 Схема связи объектов предметной области	14
3 Проектирование	15
3.1 Таблицы с описанием таблиц базы данных	15
3.2 Лингвистическое описание схемы базы данных	17
3.3 Исходный текст программы генерации схемы базы данных	21
4 Программирование	22
Заключение	23
Список использованных источников	24
Приложение А	25
Приложение Б	27

Введение

1. Постановка задачи

2. Аналитика

2.1. Описание предметной области

Геомониторинг автопарка каршеринга - это постоянная система контроля местонахождения автомобилей, которые сдаются в краткосрочную аренду. Каршеринг - это форма использования транспортных средств, при которой клиент получает временный доступ к автомобилю на ограниченный промежуток времени для передвижения по городу. Автопарк каршеринга включает все автомобили, находящиеся в собственности компании и доступные для аренды клиентами. Поскольку такие автомобили эксплуатируются каждый день, перемещаются по большой территории и используются разными водителями, возникает необходимость в непрерывном наблюдении за их местонахождением и техническим состоянием.

Основой геомониторинга является определение местоположения автомобиля в реальном времени. Для этого в каждый автомобиль устанавливается телематический терминал - устройство, предназначенное для сбора и передачи данных. Терминал оснащён спутниковым модулем, который работает одновременно с навигационными системами GPS и ГЛОНАСС, что повышает точность и надёжность определения координат.

GPS - это глобальная спутниковая навигационная система. Она работает так: над Землёй летает группа из 32 спутников на высоте примерно 20 000 километров. Каждый спутник постоянно передаёт радиосигнал, в котором содержится информация о времени отправки сигнала и о том, где именно находится спутник. Приёмник в машине ловит эти сигналы и измеряет, сколько времени сигнал шёл от спутника до приёмника. Зная скорость радиоволн (скорость света), приёмник вычисляет расстояние до каждого спутника. Чтобы точно определить своё положение на Земле, приёмнику нужно поймать сигнал минимум от трех спутников одновременно.

Координаты определяются в системе WGS-84 (World Geodetic System 1984) - это мировой стандарт координат для GPS. В этой системе положение любой точки на Земле описывается тремя числами: широта (насколько далеко от экватора), долгота (насколько далеко от нулевого меридиана) и высота. Система WGS-84 представляет Землю как эллипсоид - немного сплюснутый шар. Центр системы координат WGS-84 совпадает с центром масс Земли, а математические параметры эллипсоида подобраны так чтобы максимально точно описать форму планеты. В городе GPS определяет координаты с точностью 5-15 метров, этого достаточно для каршеринга. В России раньше использовали другие системы координат - СК-42 (система координат 1942 года, Пулково 42) и СК-95 (ПЗ-90, используется в ГЛОНАСС). Но для GPS используется именно WGS-84, потому что это единый стандарт для всего мира, и не нужно пересчитывать координаты при переезде из одной страны в другую.

Чтобы система работала лучше, в телематический терминал встроена ещё одна технология - инерциальная система навигации (ИНС). Это набор датчиков:

акселерометры (измеряют ускорение машины) и гироскопы (измеряют повороты). Когда машина заезжает в тоннель или под мост, сигнал со спутников может пропасть. В этот момент инерциальная система продолжает следить за движением машины - она знает последнюю известную точку, скорость и направление, и может примерно посчитать, где машина находится сейчас. Это называется аппроксимация координат - система делает приблизительный расчёт положения на основе данных о движении. Когда сигнал со спутников снова появляется, система GPS корректирует эту погрешность и снова начинает показывать точные координаты. Такая комбинация спутниковой навигации и инерциальных датчиков позволяет непрерывно отслеживать машину даже в сложных городских условиях.

Собранные данные о местоположении автомобиля передаются на сервер компании по сотовым сетям связи 4G (LTE). Для этого в терминале стоит SIM-карта с IoT-тарифом. IoT (Internet of Things) - это когда разные устройства подключены к интернету и автоматически обмениваются данными без участия человека. Например, холодильник может сам заказывать продукты, когда они заканчиваются. В каршеринге телематический терминал - это IoT-устройство: он сам собирает информацию где машина, с какой скоростью едет, сколько топлива осталось и автоматически отправляет эти данные на сервер компании. IoT-тарифы сотовых операторов специально сделаны для таких устройств: они работают круглосуточно, стоят дешево, работают в любом месте города без разрывов связи и очень надёжные. Данные отправляются либо раз в какое то количество времени, либо когда происходит что-то важное, например машина превысила скорость, резко затормозила, произошёл удар, началась или закончилась аренда.

В случае потери связи по мобильной сети телематический терминал сохраняет данные в свою внутреннюю память обычно от 512 МБ до 2 ГБ. Этого хватает, чтобы сохранить информацию о поездках за несколько дней. Система на сервере компании не паникует сразу, если связь с машиной пропала - она ждёт 5-10 минут, потому что связь может временно пропасть из-за помех или плохого покрытия сети. Если связь не появляется после этого времени, внешняя серверная система автоматически создаёт уведомление и отправляет его операторам. Важный момент: анализ ситуации и решение о том, что нужно действовать, принимает не сама машина, а серверная система компании. Сервер постоянно следит за всеми машинами автопарка, проверяет поступающие данные, сравнивает их с нормальными показателями и создаёт тревожные сообщения для операторов, если что-то идёт не так.

Операторы колл-центра связываются с клиентом, который использует машину, чтобы выяснить, что случилось. Если клиент не отвечает или обнаружена неисправность, на последнее известное место, где была машина, отправляется сервисная бригада. Эти специалисты ищут машину, заправляют её, заряжают или устраняют мелкие поломки.

Системой геомониторинга пользуются различные категории людей. Клиенты каршеринга через мобильное приложение видят автомобили на карте, их

координаты и уровень топлива или заряда, что позволяет выбрать подходящую машину. Операторы колл-центра используют систему для контроля текущих поездок, помощи клиентам и обработки нештатных ситуаций. Передвижные сервисные группы получают задания с указанием конкретных автомобилей и их координат для технического обслуживания. Аналитики компании используют данные для анализа загрузки автопарка, выявления зон дефицита или избытка автомобилей и оптимизации распределения машин по городу.

Ключевой процесс 1 - построение трека движения автомобиля. Телематический терминал непрерывно фиксирует координаты автомобиля и отправляет их на сервер с интервалом 10-15 секунд. Серверная система принимает каждую новую точку с координатами, временной меткой и последовательно записывает все полученные точки в базу данных, формируя трек движения. Для каждого автомобиля ведётся отдельный трек, который показывает весь маршрут за определённый период времени. По этим данным система строит линию на карте, соединяющую все точки в хронологическом порядке, что позволяет визуально увидеть, где именно ездила машина. В этом процессе участвуют несколько ролей: клиент каршеринга непосредственно управляет автомобилем во время поездки, а телематический терминал в фоновом режиме собирает координаты и отправляет их на сервер компании, оператор колл-центра в реальном времени наблюдает за движением автомобиля на карте и может отследить текущее местоположение любой машины из автопарка если поступил запрос от клиента или возникла нештатная ситуация, аналитик компании работает с архивными треками за прошедшие периоды и строит отчёты по пробегам, популярным маршрутам и зонам активности, а администратор системы следит за корректностью поступления данных от всех терминалов и настраивает параметры записи треков в базу данных. Трек сохраняется в архиве и используется для анализа: расчёта пробега, определения времени нахождения в конкретных зонах, проверки соблюдения разрешённой территории эксплуатации, расследования инцидентов и споров с клиентами. Результатом процесса является полная история перемещений каждого автомобиля с точной географической привязкой, доступная для просмотра и анализа операторами и аналитиками компании.

Ключевой процесс 2 - определение зон парковки и контроль завершения аренды. Когда клиент завершает аренду через мобильное приложение, серверная система фиксирует финальные координаты автомобиля и начинает анализ местоположения. Программа на сервере проверяет, находится ли автомобиль в пределах разрешённой зоны завершения аренды - это может быть вся территория города или определённые районы в зависимости от тарифа. Далее система определяет тип места парковки: на улице в зоне бесплатной парковки, на платной парковке, во дворе жилого дома, на территории торгового центра. Для этого сервер сравнивает координаты с заранее загруженными границами парковочных зон. Если автомобиль припаркован в запрещённом месте вне парковочной зоны, сервер отклоняет завершение аренды и уведомляет клиента о необходимости переместить автомобиль в правильное место. В этом процессе ключевую роль играет клиент каршеринга, который паркует автомобиль в

конце поездки и через мобильное приложение нажимает кнопку завершения аренды, после чего приложение показывает ему результат проверки координат и либо подтверждает завершение либо просит переставить машину в другое место, оператор колл-центра вмешивается в процесс только если у клиента возникли вопросы по парковке или система зафиксировала нарушение правил, администратор системы заранее загружает в базу данных актуальные границы разрешённых парковочных зон и обновляет их при изменении городских правил парковки, а аналитик компании изучает статистику по местам завершения аренды и выявляет зоны где скапливается слишком много автомобилей или наоборот где машин не хватает чтобы предложить корректировки тарифов или бонусов за парковку в нужных местах. Также система проверяет расстояние от припаркованного автомобиля до других машин автопарка - если в одном месте скапливается слишком много автомобилей, система может предложить клиенту скидку за парковку в дефицитной зоне или наоборот, доплату за парковку в зоне избытка. После успешной проверки система фиксирует координаты парковки, делает запись в базу данных и отображает автомобиль на карте как доступный для следующего клиента. Результатом процесса является корректное размещение автомобиля на парковке в разрешённой зоне с зафиксированными координатами, что обеспечивает равномерное распределение автопарка по городу и соблюдение правил парковки.

2.2. Цели функционирования базы данных

1. Хранение треков движения автомобилей. База данных должна сохранять все координаты автомобилей с временными метками, чтобы система могла строить треки движения.
2. Хранение зон парковки. База данных должна хранить границы зон парковки, чтобы система могла проверять правильность завершения аренды и предлагать клиентам корректные места для парковки.
3. Хранение тревожных событий связанных с автомобилем. База данных должна фиксировать все инциденты, такие как превышение скорости, резкое торможение, удар, попытки угона, чтобы операторы могли оперативно реагировать на нештатные ситуации.
4. Хранение обращений персонала к данным геомониторинга. База данных должна сохранять все запросы операторов, администраторов к данным геомониторинга, чтобы можно было отслеживать кто и когда обращался к информации о местоположении автомобилей и какие данные запрашивал.

2.3. Сущности предметной области и их атрибуты

1. **Автомобиль** – транспортное средство, которым владеет каршеринг.

- Гос номер – государственный регистрационный знак автомобиля
 - Марка – производитель автомобиля
 - Модель – название модели автомобиля
 - Год выпуска – год производства автомобиля
 - VIN-номер – уникальный идентификационный номер транспортного средства
 - Текущий статус – состояние автомобиля (свободен, в аренде, на обслуживании, неисправен)
2. **Трек движения** – траектория перемещения автомобиля внутри одной поездки.
- Время начала трека – момент начала записи маршрута движения
 - Время окончания трека – момент завершения записи маршрута движения
 - Статус трека – состояние записи (активный, завершён, архивный)
 - Вид трека – категория маршрута (пользовательский, сервисный)
3. **Координаты точки трека** – географическая точка с временной меткой на маршруте автомобиля.
- Широта – географическая широта в системе координат WGS-84
 - Долгота – географическая долгота в системе координат WGS-84
 - Скорость движения – скорость автомобиля в момент измерения в км/ч
 - Источник данных – система определения координат (GPS, инерциальная система)
4. **Зона парковки** – территория для завершения аренды и парковки автомобилей.
- Название – наименование зоны парковки
 - Тип зоны – категория парковки (бесплатная, платная, запрещённая)
 - Район города – административный район расположения зоны
 - Максимальное количество автомобилей – лимит машин которые могут одновременно находиться в зоне
 - Статус активности зоны – признак доступности зоны для использования
5. **Координаты зоны парковки** – вершины полигона определяющего границы зоны парковки на карте.
- Номер вершины по порядку – порядковый номер точки в контуре полигона

- Широта – географическая широта вершины в системе координат WGS-84
 - Долгота – географическая долгота вершины в системе координат WGS-84
6. **Тревожное событие** – инцидент или нештатная ситуация зафиксированная системой мониторинга автомобиля.
- Тип события – категория инцидента (потеря связи, превышение скорости, резкое торможение, удар, попытка угона, выезд за границу)
 - Широта – географическая широта места события
 - Долгота – географическая долгота места события
 - Описание события – текстовая информация о произошедшем инциденте
 - Статус обработки – состояние реагирования на событие (новое, в работе, закрыто)
7. **Сотрудник** – работник компании имеющий доступ к системе геомониторинга автопарка.
- ФИО – полное имя сотрудника (фамилия, имя, отчество)
 - Логин – учётная запись для входа в систему
 - Статус – текущее состояние сотрудника (активен, в отпуске, уволен, заблокирован)
 - Должность – занимаемая позиция в компании
8. **Тип сотрудника** – категория работника определяющая его роль и права доступа.
- Название типа – наименование категории (оператор колл-центра, администратор системы, сервисный специалист, аналитик)
 - Описание – текстовое пояснение роли и функций данного типа сотрудника
9. **Обращения к геомониторингу** – обращение сотрудника к системе для получения данных о местоположении.
- Тип запроса – категория запрашиваемых данных (текущие координаты, история событий)
 - Цель запроса – причина обращения к данным о местоположении автомобиля

2.4. Перечень ролей

1. **Клиент каршеринга** - человек, который арендует автомобиль. В приложении он может найти машину на карте, начать поездку и потом её закончить.
2. **Оператор колл-центра** - сотрудник, который следит за всеми машинами на карте, помогает клиентам с вопросами по аренде и реагирует на нештатные ситуации.
3. **Сервисная бригада** - специалисты, которые выполняют техническое обслуживание, перегонку автомобилей. Они получают задания от системы, которая указывает конкретные машины и их координаты для обслуживания, заправки или ремонта.
4. **Администратор системы** - человек, который отвечает за настройку и поддержку системы геомониторинга. Он загружает данные о зонах парковки, следит за корректностью поступления данных от терминалов и управляет правами доступа сотрудников.

1. Автомобиль находится в зоне парковки
2. Зона парковки ограничивается координатами зоны парковки
3. Автомобиль передвигается по треку движения
4. Координаты точки трека составляют трек движения
5. Автомобиль определяется при обращении к системе геомониторинга
6. Автомобиль провоцирует тревожное событие
7. Тип сотрудника классифицирует сотрудника
8. Сотрудник реагирует на тревожное событие

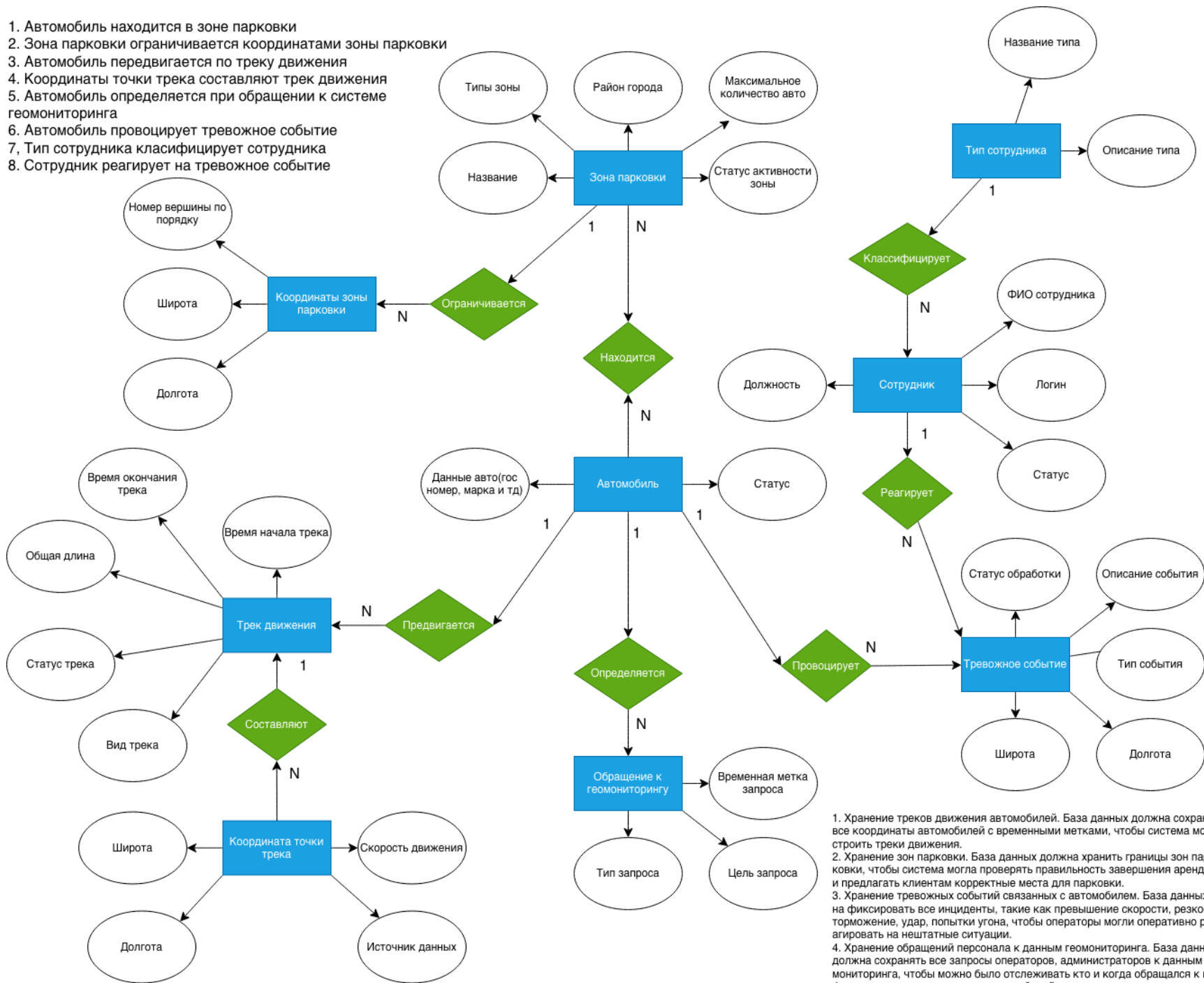


Рис. 1: ER-диаграмма

2.5. Схема связи объектов предметной области



Рис. 2: Схема связи объектов предметной области

3. Проектирование

3.1. Таблицы с описанием таблиц базы данных

В таблицах 1–9 представлены атрибуты таблиц базы данных геомониторинга. Каждая таблица с описанием содержит колонки: номер по порядку, имя атрибута на русском и английском языках, тип атрибута, при необходимости со значением размера, значение по умолчанию, признак ключа при наличии, для внешнего ключа ссылка на таблицу, примечание.

Таблица 1: Типы сотрудников

Тип сотрудника – employee_type							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	тип_сотрудника_id	employee_type_id	INT	-	PK	-	NOT NULL
2	название_типа	name	VARCHAR(50)	-	UQ	-	NOT NULL
3	описание	description	TEXT	NULL	-	-	NULL

Таблица 2: Таблица сотрудников

Сотрудник – employee							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	сотрудник_id	employee_id	INT	-	PK	-	NOT NULL
2	тип_сотрудника_id	employee_type_id	INT	-	FK	empl_type	NOT NULL
3	фio	full_name	VARCHAR(255)	-	-	-	NOT NULL
4	логин	login	VARCHAR(50)	-	UQ	-	NOT NULL
5	статус	status	VARCHAR(30)	-	-	-	NOT NULL
6	должность	position	VARCHAR(50)	-	-	-	NOT NULL

Таблица 3: Таблица зон парковки

Зона парковки – parking_zone							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	зона_парковки_id	parking_zone_id	INT	-	PK	-	NOT NULL
2	название	name	VARCHAR(100)	-	-	-	NOT NULL
3	тип_зоны	zone_type	VARCHAR(30)	-	-	-	NOT NULL
4	район_города	city_district	VARCHAR(100)	-	-	-	NOT NULL
5	макс_авто	max_cars	INT	-	-	-	NOT NULL
6	статус	active_status	VARCHAR(30)	-	-	-	NOT NULL

Таблица 4: Координаты зон парковки

Координаты зоны парковки – parking_zone_point							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	координаты_зоны_id	parking_zone_point_id	INT	-	PK	-	NOT NULL
2	зона_парковки_id	parking_zone_id	INT	-	FK	park_zone	NOT NULL
3	номер_вершины	vertex_number	INT	-	-	-	NOT NULL
4	широта	latitude	NUMERIC(9,6)	-	-	-	NOT NULL
5	долгота	longitude	NUMERIC(9,6)	-	-	-	NOT NULL

Таблица 5: Таблица автомобилей

Автомобиль – car							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	авто_id	car_id	INT	-	PK	-	NOT NULL
2	зона_парковки_id	parking_zone_id	INT	NULL	FK	park_zone	NULL
3	гос_номер	reg_number	VARCHAR(20)	-	UQ	-	NOT NULL
4	марка	brand	VARCHAR(100)	-	-	-	NOT NULL
5	модель	model	VARCHAR(100)	-	-	-	NOT NULL
6	год_выпуска	manufacture_year	INT	-	-	-	NOT NULL
7	vin_номер	vin	VARCHAR(17)	-	UQ	-	NOT NULL
8	статус	current_status	VARCHAR(30)	-	-	-	NOT NULL

Таблица 6: Таблица треков движения

Трек движения – track							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	трек_id	track_id	INT	-	PK	-	NOT NULL
2	авто_id	car_id	INT	-	FK	car	NOT NULL
3	время_начала	start_time	TIMESTAMP	-	-	-	NOT NULL
4	время_конца	end_time	TIMESTAMP	NULL	-	-	NULL
5	статус_трека	track_status	VARCHAR(30)	-	-	-	NOT NULL
6	вид_трека	track_kind	VARCHAR(30)	-	-	-	NOT NULL

Таблица 7: Точки трека движения

Координаты точки трека – track_point							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	координаты_точки_id	track_point_id	INT	-	PK	-	NOT NULL
2	трек_id	track_id	INT	-	FK	mov_track	NOT NULL
3	авто_id	car_id	INT	-	FK	car	NULL
4	широта	latitude	NUMERIC(9,6)	-	-	-	NOT NULL
5	долгота	longitude	NUMERIC(9,6)	-	-	-	NOT NULL
6	скорость	speed_kmh	NUMERIC(6,2)	NULL	-	-	NULL
7	источник_данных	data_source	VARCHAR(30)	-	-	-	NOT NULL

Таблица 8: Таблица тревожных событий

Тревожное событие – alert_event							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	событие_id	alert_event_id	INT	-	PK	-	NOT NULL
2	авто_id	car_id	INT	-	FK	car	NOT NULL
3	сотрудник_id	employee_id	INT	NULL	FK	employee	NULL
4	тип_события	event_type	VARCHAR(50)	-	-	-	NOT NULL
5	широта	latitude	NUMERIC(9,6)	NULL	-	-	NULL
6	долгота	longitude	NUMERIC(9,6)	NULL	-	-	NULL
7	описание	description	TEXT	NULL	-	-	NULL
8	статус_обработки	process_status	VARCHAR(30)	-	-	-	NOT NULL

Таблица 9: Таблица обращений

Обращение к геомониторингу – geo_request							
№	Название RU	Название EN	Тип атрибута	Def	Тип ключа	Ссылка	Прим.
1	запрос_id	geo_request_id	INT	-	PK	-	NOT NULL
2	сотрудник_id	employee_id	INT	-	FK	employee	NOT NULL
3	авто_id	car_id	INT	-	FK	car	NOT NULL
4	тип_запроса	request_type	VARCHAR(50)	-	-	-	NOT NULL
5	цель_запроса	request_goal	TEXT	-	-	-	NOT NULL

3.2. Лингвистическое описание схемы базы данных

1. Тип сотрудника (тип_сотрудника_id(INT), название_типа(VARCHAR(50)), описание(TEXT)).
2. Сотрудник (сотрудник_id(INT), тип_сотрудника_id(INT), фιο(VARCHAR(255)), логин(VARCHAR(50)), статус(VARCHAR(30)), должность(VARCHAR(50))).
3. Зона парковки (зона_парковки_id(INT), название(VARCHAR(100)), тип_зоны(VARCHAR(30)), район_города(VARCHAR(100)), макс_авто(INT), статус(VARCHAR(30))).
4. Координаты зоны парковки (координаты_зоны_id(INT), зона_парковки_id(INT), номер_вершины(INT), широта(NUMERIC(9,6)), долгота(NUMERIC(9,6))).
5. Автомобиль (авто_id(INT), зона_парковки_id(INT), гос_номер(VARCHAR(20)), марка(VARCHAR(100)), модель(VARCHAR(100)), год_выпуска(INT), vin_номер(VARCHAR(17)), статус(VARCHAR(30))).
6. Трек движения (трек_id(INT), авто_id(INT), время_начала(TIMESTAMP), время_конца(TIMESTAMP), статус_трека(VARCHAR(30)), вид_трека(VARCHAR(30))).

7. Координаты точки трека (координаты_точки_id(INT), трек_id(INT), широта(NUMERIC(9,6)), долгота(NUMERIC(9,6)), скорость(NUMERIC(6,2)), источник_данных(VARCHAR(30))).
8. Тревожное событие (событие_id(INT), авто_id(INT), сотрудник_id(INT), тип_события(VARCHAR(50)), широта(NUMERIC(9,6)), долгота(NUMERIC(9,6)), описание(TEXT), статус_обработки(VARCHAR(30))).
9. Обращения к геомониторингу (запрос_id(INT), сотрудник_id(INT), авто_id(INT), тип_запроса(VARCHAR(50)), цель_запроса(TEXT)).



Рис. 3: Уровни заполнения базы данных

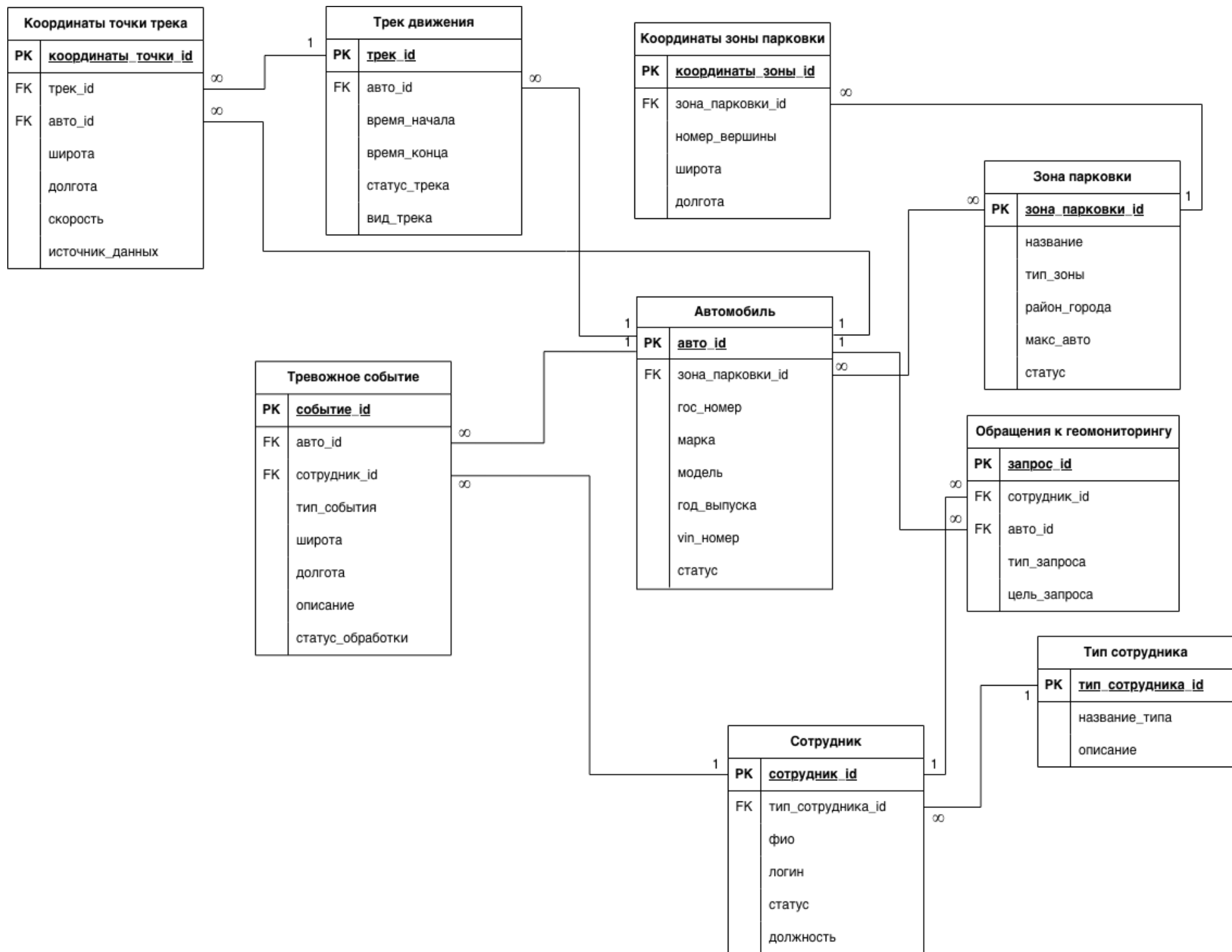


Рис. 4: Графическое описание схемы базы данных на русском языке

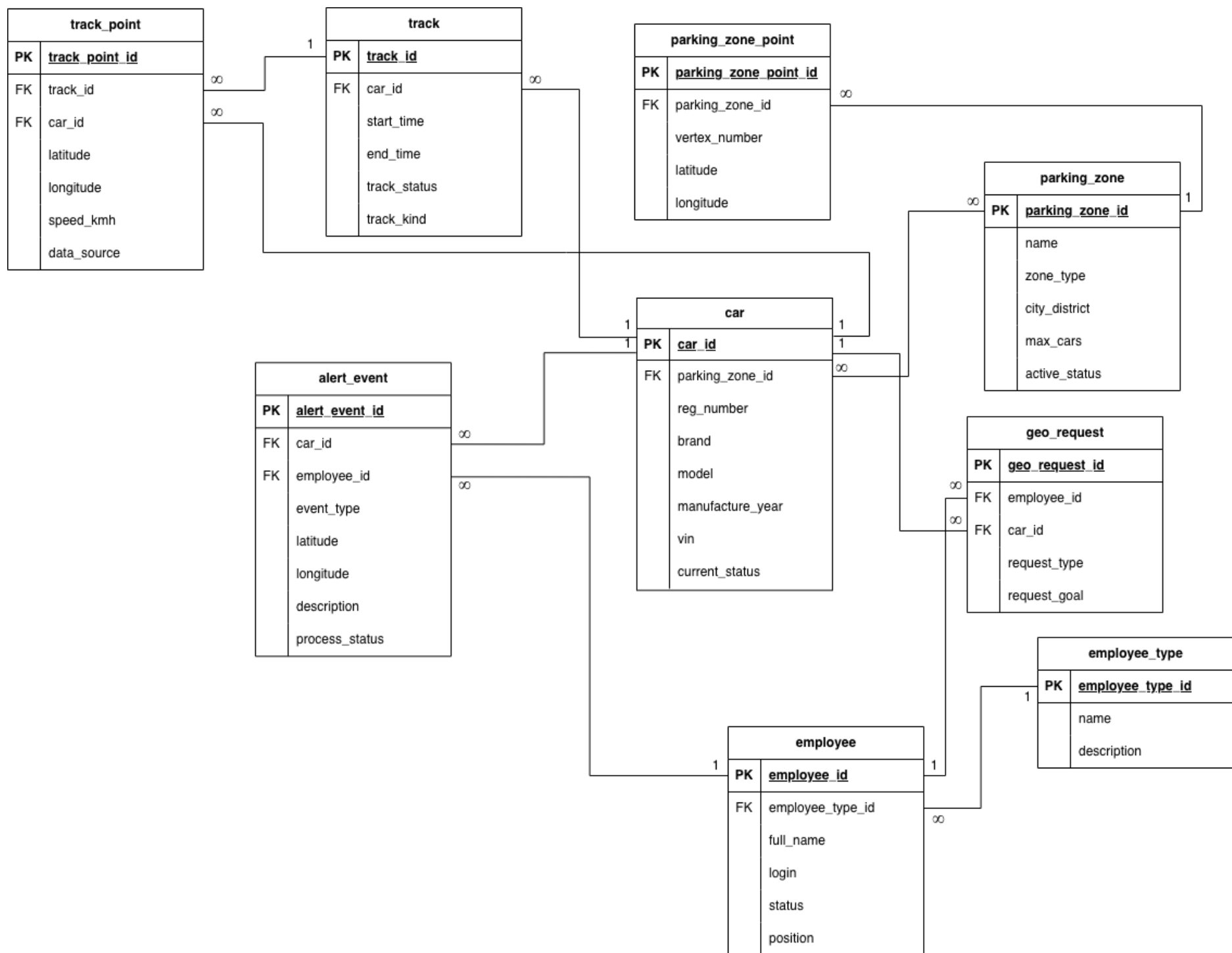


Рис. 5: Графическое описание схемы базы данных на английском языке

3.3. Исходный текст программы генерации схемы базы данных

База данных создана средствами СУБД PostgreSQL с использованием SQL-скриптов, приведенных в [приложении А](#) (схема на рис. 3).

Наполнение базы синтетическими данными (320225 записей в 9 таблицах) выполнено утилитой на языке Go 1.25.4 с использованием драйвера pgx. Таблицы-справочники заполнены статическими данными, а основные сущности сгенерированы алгоритмически с учетом логических связей в [приложении Б](#).

4. Программирование

Заключение

Список литературы

Приложение А «Исходный код программы генерации схемы базы данных»

```
1 CREATE TABLE employee_type (  
2     employee_type_id  SERIAL PRIMARY KEY,  
3     name              VARCHAR(50) NOT NULL UNIQUE,  
4     description       TEXT  
5 );  
6  
7 CREATE TABLE employee (  
8     employee_id       SERIAL PRIMARY KEY,  
9     employee_type_id  INT NOT NULL REFERENCES  
10    employee_type(employee_type_id),  
11    full_name         VARCHAR(255) NOT NULL,  
12    login             VARCHAR(50) NOT NULL UNIQUE,  
13    status            VARCHAR(30) NOT NULL,  
14    position          VARCHAR(50) NOT NULL  
15 );  
16  
17 CREATE TABLE parking_zone (  
18     parking_zone_id  SERIAL PRIMARY KEY,  
19     name             VARCHAR(100) NOT NULL,  
20     zone_type        VARCHAR(30) NOT NULL,  
21     city_district    VARCHAR(100) NOT NULL,  
22     max_cars         INT NOT NULL,  
23     active_status    VARCHAR(30) NOT NULL  
24 );  
25  
26 CREATE TABLE parking_zone_point (  
27     parking_zone_point_id SERIAL PRIMARY KEY,  
28     parking_zone_id      INT NOT NULL REFERENCES  
29     parking_zone(parking_zone_id) ON DELETE CASCADE,  
30     vertex_number        INT NOT NULL,  
31     latitude             NUMERIC(9,6) NOT NULL,  
32     longitude            NUMERIC(9,6) NOT NULL,  
33     UNIQUE (parking_zone_id, vertex_number)  
34 );  
35  
36 CREATE TABLE car (  
37     car_id           SERIAL PRIMARY KEY,  
38     parking_zone_id  INT REFERENCES  
39     parking_zone(parking_zone_id),  
40     reg_number       VARCHAR(20) NOT NULL UNIQUE,  
41     brand            VARCHAR(100) NOT NULL,  
42     model            VARCHAR(100) NOT NULL,  
43     manufacture_year INT NOT NULL,  
44     vin              VARCHAR(17) NOT NULL UNIQUE,  
45     current_status   VARCHAR(30) NOT NULL  
46 );  
47  
48 CREATE TABLE track (  
49     track_id         SERIAL PRIMARY KEY,
```

```

47     car_id          INT NOT NULL REFERENCES car(car_id) ON
      DELETE CASCADE,
48     start_time      TIMESTAMP NOT NULL,
49     end_time        TIMESTAMP,
50     track_status    VARCHAR(30) NOT NULL,
51     track_kind      VARCHAR(30) NOT NULL
52 );
53
54 CREATE TABLE track_point (
55     track_point_id   SERIAL PRIMARY KEY,
56     track_id         INT REFERENCES track(track_id) ON DELETE
      CASCADE, -- NULL разрешён
57     car_id          INT NOT NULL REFERENCES car(car_id) ON
      DELETE CASCADE,
58     latitude        NUMERIC(9,6) NOT NULL,
59     longitude       NUMERIC(9,6) NOT NULL,
60     speed_kmh       NUMERIC(6,2),
61     data_source     VARCHAR(30) NOT NULL
62 );
63
64 CREATE TABLE alert_event (
65     alert_event_id   SERIAL PRIMARY KEY,
66     car_id          INT NOT NULL REFERENCES car(car_id) ON
      DELETE CASCADE,
67     employee_id     INT REFERENCES employee(employee_id),
68     event_type      VARCHAR(50) NOT NULL,
69     latitude        NUMERIC(9,6),
70     longitude       NUMERIC(9,6),
71     description     TEXT,
72     process_status  VARCHAR(30) NOT NULL
73 );
74
75 CREATE TABLE geo_request (
76     geo_request_id   SERIAL PRIMARY KEY,
77     employee_id     INT NOT NULL REFERENCES
      employee(employee_id),
78     car_id          INT NOT NULL REFERENCES car(car_id),
79     request_type    VARCHAR(50) NOT NULL,
80     request_goal     TEXT NOT NULL
81 );

```

Listing 1: SQL-скрипт для создания схемы базы данных

Приложение Б «Исходный код программы заполнения базы данных тестовыми данными»

```
1 func truncateAll(ctx context.Context, tx pgx.Tx) error {
2     _, err := tx.Exec(ctx, `
3         TRUNCATE TABLE
4             geo_request,
5             alert_event,
6             track_point,
7             track,
8             car,
9             parking_zone_point,
10            employee,
11            parking_zone,
12            employee_type
13     RESTART IDENTITY CASCADE
14 `)
15     if err != nil {
16         return fmt.Errorf("truncate tables: %w", err)
17     }
18     return nil
19 }
```

Listing 2: Функция для очистки всех таблиц

```
1 func seedEmployeeTypes(ctx context.Context, tx pgx.Tx, r
2     *rand.Rand, count int) ([]int, error) {
3     names := []string{"operator", "dispatcher", "manager",
4         "technician", "analyst"}
5     ids := make([]int, 0, count)
6
7     for i := 0; i < count; i++ {
8         var id int
9         err := tx.QueryRow(ctx,
10             `INSERT INTO employee_type(name, description)
11             VALUES ($1, $2)
12             RETURNING employee_type_id`,
13             fmt.Sprintf("%s_%d", names[i%len(names)], i+1),
14             fmt.Sprintf("generated employee type %d", i+1),
15         ).Scan(&id)
16         if err != nil {
17             return nil, fmt.Errorf("insert employee_type[%d]: %w", i,
18                 err)
19         }
20         ids = append(ids, id)
21     }
22     return ids, nil
23 }
```

Listing 3: Функция для заполнения таблицы employee_type

```

1 func seedParkingZones(ctx context.Context, tx pgx.Tx, r
    *rand.Rand, count int) ([]int, error) {
2     zoneTypes := []string{"city", "suburban", "hub"}
3     zoneStatuses := []string{"active", "maintenance", "limited"}
4     ids := make([]int, 0, count)
5
6     for i := 0; i < count; i++ {
7         var id int
8         err := tx.QueryRow(ctx,
9             `INSERT INTO parking_zone(name, zone_type, city_district,
10              max_cars, active_status)
11              VALUES ($1, $2, $3, $4, $5)
12              RETURNING parking_zone_id`,
13             fmt.Sprintf("Zone-%03d", i+1),
14             zoneTypes[r.Intn(len(zoneTypes))],
15             fmt.Sprintf("district-%d", 1+r.Intn(8)),
16             20+r.Intn(150),
17             zoneStatuses[r.Intn(len(zoneStatuses))],
18         ).Scan(&id)
19         if err != nil {
20             return nil, fmt.Errorf("insert parking_zone[%d]: %w", i, err)
21         }
22         ids = append(ids, id)
23     }
24     return ids, nil
25 }

```

Listing 4: Функция для заполнения таблицы parking_zone

```

1 func seedEmployees(ctx context.Context, tx pgx.Tx, r *rand.Rand,
    employeeTypeIDs []int, count int) ([]int, error) {
2     statuses := []string{"active", "vacation", "inactive"}
3     positions := []string{"junior", "middle", "senior", "lead"}
4     ids := make([]int, 0, count)
5
6     for i := 0; i < count; i++ {
7         var id int
8         err := tx.QueryRow(ctx,
9             `INSERT INTO employee(employee_type_id, full_name, login,
10              status, position)
11              VALUES ($1, $2, $3, $4, $5)
12              RETURNING employee_id`,
13             employeeTypeIDs[r.Intn(len(employeeTypeIDs))],
14             fmt.Sprintf("Employee %04d", i+1),
15             fmt.Sprintf("user_%04d", i+1),
16             statuses[r.Intn(len(statuses))],
17             positions[r.Intn(len(positions))],
18         ).Scan(&id)
19         if err != nil {
20             return nil, fmt.Errorf("insert employee[%d]: %w", i, err)
21         }
22         ids = append(ids, id)
23     }
24 }

```

```

23 return ids, nil
24 }

```

Listing 5: Функция для заполнения таблицы employee

```

1 func seedParkingZonePoints(ctx context.Context, tx pgx.Tx, r
  *rand.Rand, parkingZoneIDs []int, pointsPerZone int) error {
2   for _, zoneID := range parkingZoneIDs {
3     for pointNo := 1; pointNo <= pointsPerZone; pointNo++ {
4       _, err := tx.Exec(ctx,
5         `INSERT INTO parking_zone_point(parking_zone_id,
6         vertex_number, latitude, longitude)
7         VALUES ($1, $2, $3, $4)`,
8         zoneID,
9         pointNo,
10        59.85+r.Float64()*0.2,
11        30.15+r.Float64()*0.35,
12      )
13      if err != nil {
14        return fmt.Errorf("insert parking_zone_point(zone=%d,
15        vertex=%d): %w", zoneID, pointNo, err)
16      }
17    }
18  }
19  return nil
20 }

```

Listing 6: Функция для заполнения таблицы parking_zone_point

```

1 func seedCars(ctx context.Context, tx pgx.Tx, r *rand.Rand,
  parkingZoneIDs []int, count int) ([]int, error) {
2   brands := []string{"VW", "Ford", "Hyundai", "Kia", "Lada",
3     "Toyota"}
4   models := []string{"A", "B", "C", "D", "E"}
5   carStatuses := []string{"available", "busy", "maintenance",
6     "offline"}
7   ids := make([]int, 0, count)
8
9   for i := 0; i < count; i++ {
10    var id int
11    err := tx.QueryRow(ctx,
12      `INSERT INTO car(parking_zone_id, reg_number, brand, model,
13      manufacture_year, vin, current_status)
14      VALUES ($1, $2, $3, $4, $5, $6, $7)
15      RETURNING car_id`,
16      parkingZoneIDs[r.Intn(len(parkingZoneIDs))],
17      fmt.Sprintf("A%03dAA%03d", i%1000, 100+(i%799)),
18      brands[r.Intn(len(brands))],
19      models[r.Intn(len(models))],
20      2012+r.Intn(14),
21      fmt.Sprintf("VIN%014d", i+1),
22      carStatuses[r.Intn(len(carStatuses))],
23    ).Scan(&id)
24    if err != nil {

```

```

22     return nil, fmt.Errorf("insert car[%d]: %w", i, err)
23 }
24 ids = append(ids, id)
25 }
26 return ids, nil
27 }

```

Listing 7: Функция для заполнения таблицы car

```

1 func seedAlertEvents(ctx context.Context, tx pgx.Tx, r *rand.Rand,
  carIDs, employeeIDs []int, count int) error {
2     eventTypes := []string{"speeding", "zone_leave", "panic_button",
  "signal_lost"}
3     processStatuses := []string{"new", "in_progress", "closed"}
4
5     for i := 0; i < count; i++ {
6         _, err := tx.Exec(ctx,
7             `INSERT INTO alert_event(car_id, employee_id, event_type,
  latitude, longitude, description, process_status)
8             VALUES ($1, $2, $3, $4, $5, $6, $7)`,
9             carIDs[r.Intn(len(carIDs))],
10            employeeIDs[r.Intn(len(employeeIDs))],
11            eventTypes[r.Intn(len(eventTypes))],
12            59.85+r.Float64()*0.2,
13            30.15+r.Float64()*0.35,
14            fmt.Sprintf("generated alert event %d", i+1),
15            processStatuses[r.Intn(len(processStatuses))],
16        )
17        if err != nil {
18            return fmt.Errorf("insert alert_event[%d]: %w", i, err)
19        }
20    }
21    return nil
22 }

```

Listing 8: Функция для заполнения таблицы alert_event

```

1 func seedGeoRequests(ctx context.Context, tx pgx.Tx, r *rand.Rand,
  employeeIDs, carIDs []int, count int) error {
2     requestTypes := []string{"tracking", "history", "inspection",
  "incident_review"}
3
4     for i := 0; i < count; i++ {
5         _, err := tx.Exec(ctx,
6             `INSERT INTO geo_request(employee_id, car_id, request_type,
  request_goal)
7             VALUES ($1, $2, $3, $4)`,
8             employeeIDs[r.Intn(len(employeeIDs))],
9             carIDs[r.Intn(len(carIDs))],
10            requestTypes[r.Intn(len(requestTypes))],
11            fmt.Sprintf("generated request goal %d", i+1),
12        )
13        if err != nil {
14            return fmt.Errorf("insert geo_request[%d]: %w", i, err)

```

```

15     }
16 }
17 return nil
18 }

```

Listing 9: Функция для заполнения таблицы geo_request

```

1 func seedTrackPoints(
2     ctx context.Context,
3     tx pgx.Tx,
4     r *rand.Rand,
5     tracks []trackRef,
6     pointsPerTrack int,
7     pointsPerCarWithoutTracks int,
8 ) error {
9     dataSources := []string{"gps", "glonass", "manual"}
10
11     for _, tr := range tracks {
12         for p := 0; p < pointsPerTrack; p++ {
13             _, err := tx.Exec(ctx,
14                 `INSERT INTO track_point(track_id, car_id, latitude,
15                     longitude, speed_kmh, data_source)
16                     VALUES ($1, $2, $3, $4, $5, $6)`,
17                     tr.ID,
18                     tr.CarID,
19                     59.85+r.Float64()*0.2,
20                     30.15+r.Float64()*0.35,
21                     float64(10+r.Intn(110)),
22                     dataSources[r.Intn(len(dataSources))],
23                 )
24             if err != nil {
25                 return fmt.Errorf("insert track_point(track=%d, p=%d):
26                     %w", tr.ID, p, err)
27             }
28         }
29
30         for p := 0; p < pointsPerCarWithoutTracks; p++ {
31             _, err := tx.Exec(ctx,
32                 `INSERT INTO track_point(track_id, car_id, latitude,
33                     longitude, speed_kmh, data_source)
34                     VALUES ($1, $2, $3, $4, $5, $6)`,
35                     nil,
36                     tr.CarID,
37                     59.85+r.Float64()*0.2,
38                     30.15+r.Float64()*0.35,
39                     float64(10+r.Intn(110)),
40                     dataSources[r.Intn(len(dataSources))],
41                 )
42             if err != nil {
43                 return fmt.Errorf("insert track_point(car=%d, p=%d)
44                     without track: %w", tr.CarID, p, err)
45             }
46         }
47     }
48 }

```

```

43 }
44
45 return nil
46 }

```

Listing 10: Функция для заполнения таблицы track_point

```

1 var reportTables = []string{
2     "employee_type",
3     "parking_zone",
4     "employee",
5     "parking_zone_point",
6     "car",
7     "track",
8     "alert_event",
9     "geo_request",
10    "track_point",
11 }
12
13 func buildReport(ctx context.Context, pool *pgxpool.Pool)
14    (*Report, error) {
15    report := &Report{TableCounts: make(map[string]int,
16        len(reportTables))}
17
18    for _, table := range reportTables {
19        var count int
20        if err := pool.QueryRow(ctx, fmt.Sprintf("SELECT COUNT(*) FROM
21            %s", table)).Scan(&count); err != nil {
22            return nil, fmt.Errorf("count %s: %w", table, err)
23        }
24        report.TableCounts[table] = count
25        report.TotalRows += count
26    }
27
28    return report, nil
29 }

```

Listing 11: Список таблиц для генерации отчета

```

1 func Seed(ctx context.Context, pool *pgxpool.Pool) error {
2     _, err := SeedWithReport(ctx, pool, DefaultConfig())
3     return err
4 }
5
6 func SeedWithReport(ctx context.Context, pool *pgxpool.Pool, cfg
7     Config) (*Report, error) {
8     r := rand.New(rand.NewSource(time.Now().UnixNano()))
9
10    tx, err := pool.Begin(ctx)
11    if err != nil {
12        return nil, fmt.Errorf("begin transaction: %w", err)
13    }
14    defer tx.Rollback(ctx)

```



```

15  if err = truncateAll(ctx, tx); err != nil {
16      return nil, err
17  }
18
19  // уровень 1
20  employeeTypeIDs, err := seedEmployeeTypes(ctx, tx, r,
    cfg.EmployeeTypes)
21  if err != nil {
22      return nil, err
23  }
24
25  parkingZoneIDs, err := seedParkingZones(ctx, tx, r,
    cfg.ParkingZones)
26  if err != nil {
27      return nil, err
28  }
29
30  // уровень 2
31  employeeIDs, err := seedEmployees(ctx, tx, r, employeeTypeIDs,
    cfg.Employees)
32  if err != nil {
33      return nil, err
34  }
35
36  if err = seedParkingZonePoints(ctx, tx, r, parkingZoneIDs,
    cfg.ZonePointsPerZone); err != nil {
37      return nil, err
38  }
39
40  carIDs, err := seedCars(ctx, tx, r, parkingZoneIDs, cfg.Cars)
41  if err != nil {
42      return nil, err
43  }
44
45  // уровень 3
46  tracks, err := seedTracks(ctx, tx, r, carIDs, cfg.TracksPerCar)
47  if err != nil {
48      return nil, err
49  }
50
51  if err = seedAlertEvents(ctx, tx, r, carIDs, employeeIDs,
    cfg.AlertEvents); err != nil {
52      return nil, err
53  }
54
55  if err = seedGeoRequests(ctx, tx, r, employeeIDs, carIDs,
    cfg.GeoRequests); err != nil {
56      return nil, err
57  }
58
59  // уровень 4
60  if err = seedTrackPoints(ctx, tx, r, tracks,
    cfg.TrackPointsPerTrack); err != nil {

```

```
61     return nil, err
62 }
63
64 if err = tx.Commit(ctx); err != nil {
65     return nil, fmt.Errorf("commit transaction: %w", err)
66 }
67
68 return buildReport(ctx, pool)
69 }
```

Listing 12: Функция для заполнения БД